

Spotify ML project Report

Tony Lai, Ian Jang

May 2025

1 Introduction

Spotify has become an integral part of all our lives; it is the number one music platform in the US. One important aspect of Spotify that we all enjoy is making playlists. In fact, as of 2018, over 4 billion playlists have been created. As daily users, we wanted to work on something that could possibly improve our experience as users, such as the playlist recommendation algorithm. The Spotify Million Playlists dataset is public dataset consisting of 1,000,000 playlists, sampled from these public playlists on Spotify. The goal is simple: given a seed playlist/title or a set number of tracks, predict and output the recommendation tracks that may fit in with the input.

We thus framed the project around 2 questions: (1) *Can we discover coherent "vibe" clusters in the Spotify Million playlist database*, and (2) *Given a partial playlist, which specific tracks best extend it so that the continuation remains inside its discovered vibe cluster?* These questions split the work into an unsupervised "discovery phase" (cluster the playlists themselves), followed by a supervised recommendation phase (rank candidate tracks).

2 Data Preprocessing

2.1 Spotify million playlist dataset explained

This dataset is essentially a series of playlists, each with a series of tracks, of which basic information are provided for the track. A typically playlist entry would look like this:

```
{
  "name": "musical",
  "collaborative": "false",
  "pid": 5,
  "modified_at": 1493424000,
  "num_albums": 7,
  "num_tracks": 12,
  "num_followers": 1,
```

```

"num_edits": 2,
"duration_ms": 2657366,
"num_artists": 6,
"tracks": [
  {
    "pos": 0,
    "artist_name": "Degiheugi",
    "track_uri": "spotify:track:7vqa3sDmtEaVJ2gcvxtRID",
    "artist_uri": "spotify:artist:3V2paBXEoZIAhfZRJmo2jL",
    "track_name": "Finalement",
    "album_uri": "spotify:album:2KrRMJ9z7Xjoz1Az406UML",
    "duration_ms": 166264,
    "album_name": "Dancing Chords and Fireflies"
  },
  {
    "pos": 1,
    "artist_name": "Degiheugi",
    "track_uri": "spotify:track:23E0mJiv0Z88WJPUBIPjh6",
    "artist_uri": "spotify:artist:3V2paBXEoZIAhfZRJmo2jL",
    "track_name": "Betty",
    "album_uri": "spotify:album:3lUSlvjUoHNA8IkNTqURqd",
    "duration_ms": 235534,
    "album_name": "Endless Smile"
  },
  {
    "pos": 2,
    "artist_name": "Degiheugi",
    "track_uri": "spotify:track:1vaffTCJxkyqeJY7zF9a55",
    "artist_uri": "spotify:artist:3V2paBXEoZIAhfZRJmo2jL",
    "track_name": "Some Beat in My Head",
    "album_uri": "spotify:album:2KrRMJ9z7Xjoz1Az406UML",
    "duration_ms": 268050,
    "album_name": "Dancing Chords and Fireflies"
  },
  // 8 tracks omitted
  {
    "pos": 11,
    "artist_name": "Mo' Horizons",
    "track_uri": "spotify:track:7iwx00eBzeSSSy6xfESyWN",
    "artist_uri": "spotify:artist:3tuX54dqs8LsGUvNzgrpP",
    "track_name": "Fever 99\u00b0",
    "album_uri": "spotify:album:2Fg1t2ty0SGWkVYH1FfXVf",
    "duration_ms": 364320,
    "album_name": "Come Touch The Sun"
  }
],

```

}

As one can see, only basic information such as the `artist_name`, `track_uri`, and the `track_name` are provided (these features are the easiest to understand). Thus, it means that the song's metadata is not included; that is, we know nothing about the song itself. Without these audio features, it will be impossible to cluster these songs together.

2.2 2M unique spotify songs with audio features

Our original plan was to query the Spotify API at the `audio_features` endpoint to obtain the metadata; this way, we can guarantee that this entry appears (obviously, given that this is the Spotify database itself). When setting up the spotify token, we constantly ran into 403 http errors, due to either not having enough permissions, or having the wrong redirect link (Spotify does not allow localhost links anymore). After finally succeeding in setting up the token, we now have to make the actual call to get the metadata. However, after lots of back and forths between Stackoverflow, Github, and Spotify web developer forums, we finally found out that the endpoint we needed to call had been deprecated; it seems that Spotify had removed this for any developer app created after November of 2024 (most likely due wanting to monetize their data).

With this in mind, our next best bet would either be to 1. Query an audio sample from a music API such as 7digital ~30 seconds, or 2. Find a big kaggle database dump containing metadata on enough songs to cover the Million Playlists dataset. Although option 1 seems like the intuitive, better choice, the problem with it would be the training time needed. There are approximately 700k songs in subset of just 100k playlists (of a million total); assuming each song is about 2 minutes, this would mean there would be over 84 million seconds of audio to parse through, which is simply not doable without a datacenter.

We finally settled on this specific kaggle dataset, which contains audio features for around 2 million unique songs, specifically targeting the Spotify Million Playlists dataset. This is an SQLite database and is essentially a lookup table that enriches every Spotify ID in the Million Playlist dataset. In this way, we can avoid cases of not having an audio feature entry for a specific song. Core features of this dataset include:

- Danceability: (0-1), High values representing rhythmic stability
- Energy: (0-1), Perceptual intensity (loud, bright aggressive). Segregates mellow acoustic sets from heavy metal workouts
- Loudness: (-60,60) dB, Absolute loudness averaged over the track. Highly correlated with energy but still differentiate modern-mastered pop ~5 dB from 1970s jazz (~15 dB)

- Tempo: (40-50) BPM, Raw pacing; playlists often stick to tempo bands (e.g. study lofi $\sim$ 70 BPM, workout $\sim$ 140 BPM)
- Valence: (0-1) "Positiveness," (Happy vs sad playlists)
- Acousticness: (0-1), Confidence the track is acoustic

There are a couple more features apart from this, but I handpicked these as they are easy to understand.

2.3 Merging the two datasets together

The plan is then to merge these two datasets together into one unified vector per playlist, and then feed it into the unsupervised model, of which we picked MiniBatchKMeans. The code below performs the merge:

```

1 # -----
2 # 1. Playlist vectors
3 # -----
4 num_cols = ['danceability', 'energy', 'loudness', 'speechiness',
5             'acousticness', 'instrumentalness', 'liveness',
6             'valence', 'tempo']
7 pl_vec = (
8     full_df.groupby('pid')[num_cols]
9     .agg(['mean', 'std'])
10    .fillna(0)
11 )
12 pl_vec.columns = ['_'.join(c) for c in pl_vec.columns] #
13 # flatten
14 print(pl_vec.shape) # should be ~ 100 000 playlists, 18
15 # features x 2) -> assuming 100k playlists

```

3 Baseline modeling

3.1 MiniBatchKMeans

MiniBatchKMeans is a scalable, approximate variant of the classic k-Means clustering algorithm. It is specifically designed for large datasets, where instead of repeatedly scanning the entire dataset to update centroids, it processes small, random "minibatches" of points at each iteration. This dramatically reduces both computational time and memory overhead while still converging to a good solution. For context, at around $\sim$ 100k playlists, regular k-Means would take 6 hours, while A simple gridSearch kMeans ($k \in [4, 21]$) only takes a couple minutes.

Before clustering, we applied PCA into our playlist-level feature mix `pl_vec`.

```

1 from sklearn.decomposition import PCA
2 pca = PCA(n_components=0.95, random_state=42)
3 X_red = pca.fit_transform(pl_vec.values)

```

We use PCA in order for:

- Noise reduction - by keeping only the principal components that explain 95% of the variance, we can discard noise that can hurt clustering
- Speed - clustering in 7-8 dimensional subspace is faster than the original 9-D, and would scale better should more features be added later down the line
- Interpretability - allows for closer inspection of explained-variance ratios to see whether one or two components dominate, can study which features affect the most in terms of a song's placement into a vibe cluster

After we obtain `X_red` (which should be dense array by now), we run it through `MiniBatchKMeans`.

```

1 # -----
2 # 3. Fast grid-search with MiniBatchKMeans
3 # -----
4 from sklearn.cluster import MiniBatchKMeans
5 from sklearn.metrics import silhouette_score,
   davies_bouldin_score
6
7 # Take a 10-k playlist sample for metrics -- reproducible
   with seed
8 # Subsample for evaluation speed
9 rng = np.random.default_rng(42)
10 idx_samp = rng.choice(len(X_red), size=10_000, replace=False
   ) # can modify sample size
11 X_samp = X_red[idx_samp]
12
13 best = None
14 for k in range(4, 21):
15     km = MiniBatchKMeans(
16         n_clusters=k,
17         batch_size=2048,
18         max_iter=100,
19         n_init='auto', # sklearn 1.4+ does parallel in C
20         random_state=42,
21         reassignment_ratio=0.01
22     ).fit(X_red) # full data for centroids, still very fast
23
24     # metrics on the 10-k sample only
25     # evaluate clustering quality on the 10k sample
26     labels_samp = km.labels_[idx_samp]
27     sil = silhouette_score(X_samp, labels_samp)

```

```

28     db = davies_bouldin_score(X_samp, labels_samp)
29
30     print(f"k={k:2d}    sil={sil:.4f}    DB={db:.4f}")
31
32     # track the best model
33     if best is None or sil > best[1]:
34         best = (km, sil, db)
35
36 # attach winning labels to the full playlist index
37 pl_vec["cluster"] = best[0].labels_
38
39 # # 3. K-Means grid (deprecated in favor of MiniBatchKMeans)
40 # # NOTE: this is slow, so we use MiniBatchKMeans instead
41 # from sklearn.cluster import KMeans
42 # from sklearn.metrics import silhouette_score,
43 #     davies_bouldin_score
44 # best = None
45 # for k in range(4,21):
46 #     km = KMeans(k, n_init='auto', random_state=42).fit(
47 #         X_red)
48 #     sil = silhouette_score(X_red, km.labels_)
49 #     db = davies_bouldin_score(X_red, km.labels_)
50 #     print(f"k={k:2d}    sil={sil:.3f}    DB={db:.3f}")
51 #     if best is None or sil > best[1]:
52 #         best = (km, sil, db)
53 # pl_vec['cluster'] = best[0].labels_

```

3.2 MiniBatchKMeans Results

Below are the graphs for MiniBatchKMeans at 30,000 playlists – I forgot to save them when I initially ran at 100,000 playlists, and cannot retrain that right now due to time constraints.

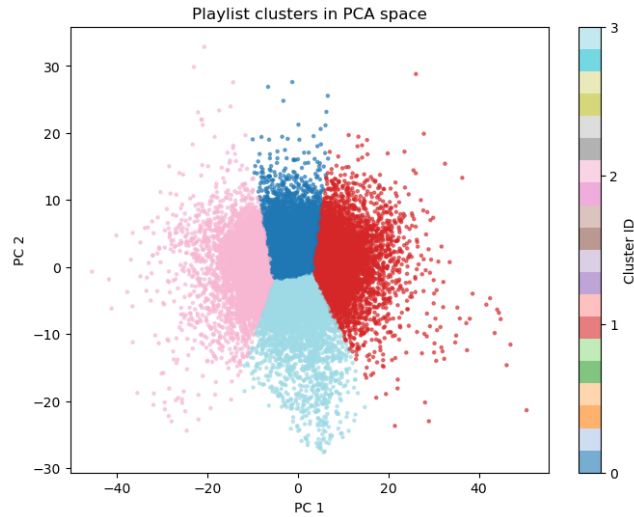


Figure 1: Playlist clusters in PCA space. Each point is one of the 30,000 playlists, projected onto the first two principal components (which together capture 95% of the variance in the 9-dimensional feature space).

Loadings on PC1 and PC2:

	PC1	PC2
danceability_mean	0.000	-0.005
danceability_std	-0.000	0.001
energy_mean	0.008	-0.007
energy_std	-0.001	0.002
loudness_mean	0.151	-0.114
loudness_std	-0.048	0.047
speechiness_mean	0.000	0.000
speechiness_std	-0.000	0.000
acousticness_mean	-0.010	0.010
acousticness_std	-0.003	0.004
instrumentalness_mean	-0.003	0.000
instrumentalness_std	-0.002	-0.002
liveness_mean	0.001	-0.000
liveness_std	0.001	-0.001
valence_mean	0.002	-0.002
valence_std	0.000	0.000
tempo_mean	0.986	-0.036
tempo_std	0.055	0.992

Figure 2: PCA loadings for PC 1 and PC 2. Each row shows how strongly a given audio-feature (means and standard deviations) contributes to the two principal components. PC 1 is almost entirely driven by mean tempo (0.986) with a secondary loudness effect (0.151), so it separates playlists from slow/quiet (left) to fast/loud (right). PC 2 is dominated by tempo variability (0.992), distinguishing playlists with uniform pacing (bottom) from those that mix slow and fast tracks (top).

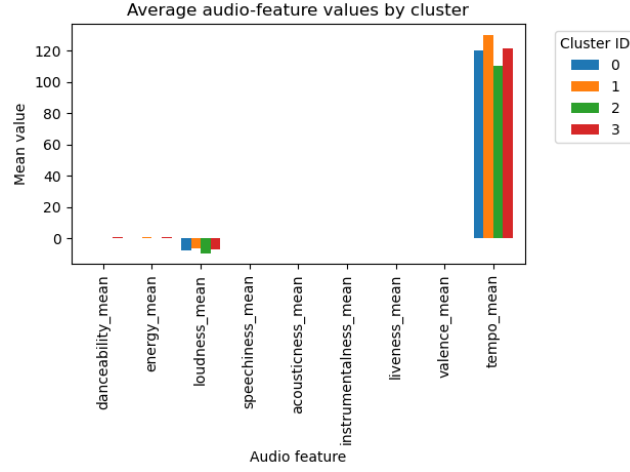


Figure 3: Cluster centroids in audio-feature space. Bars show the mean playlist-level values for nine continuous descriptors—danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, and tempo—across Clusters 0 (blue) through 3 (red). Each cluster exhibits a distinct “vibe” profile, with Cluster 1 highest in tempo, energy, and valence, and Cluster 2 most acoustic and low-energy.

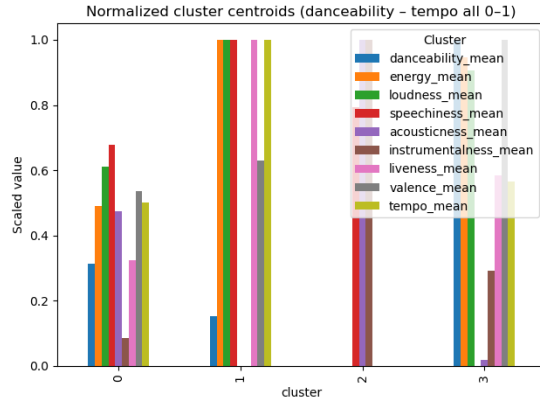


Figure 4: Normalized cluster centroids for nine mean audio features. Each bar shows a cluster’s mean feature value, scaled to $[0,1]$ across clusters so that small-range descriptors (danceability, speechiness, acousticness, etc.) are visible alongside tempo and loudness. Cluster 1 (orange) leads on energy and danceability; Cluster 2 (green) peaks on acousticness and instrumentalness; Cluster 0 (blue) is the mid-range “everyday” profile; Cluster 3 (red) sits between Clusters 0 and 1.

3.2.1 Numerical Results given different K values

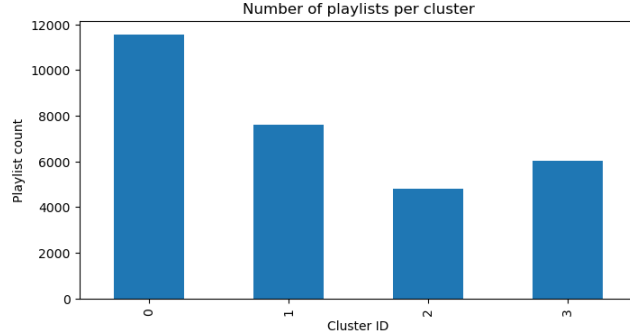


Figure 5: Bars indicate the total count of playlists assigned to Cluster 0 through Cluster 3 at the 30 000-playlist sample. Cluster 0 (blue) is the largest, capturing the majority of “everyday” playlists, while Cluster 2 (green) is the smallest, representing more niche, low-tempo/acoustic collections.

As we can see in Figure 1, the four clusters fan out into distinct wedges because PC 1 (the horizontal axis) is almost entirely a mean-tempo axis—separating slow/quiet playlists on the left from fast/loud ones on the right—while PC 2 (the vertical axis) captures tempo variability, distinguishing uniform-paced sets at the bottom from mixed-tempo mixes at the top. Together, these two tempo-based dimensions yield four naturally separated listening “vibes.”

Turning to the relative sizes of these clusters (Figure 5), we see that Cluster 0 is by far the most populous ($\sim 11\,500$ playlists), reflecting the prevalence of moderate-tempo, everyday pop/rock curations. Cluster 1 ($\sim 7\,600$) comprises the next-largest slice, corresponding to high-energy/dance sets, while Cluster 3 ($\sim 6\,000$) captures upbeat but less extreme indie/pop mixes. Cluster 2 is the smallest ($\sim 4\,800$), indicating that niche, acoustic-leaning playlists occur far less often.

3.3 Gated Recurrent Unit

A Gated Recurrent Unit (GRU) is a type of recurrent neural network designed to model sequences (for example, a playlist of track feature vectors) while mitigating the vanishing gradient problem common in vanilla RNNs (Recurrent Neural Network). Imagine you’re reading a sentence one word at a time and after each word you update a little “notebook” in your mind that captures everything you’ve read so far. That’s basically what a RNN is. At each time step t , it takes a new input x_t (e.g. the feature vector of track t) and the previous hidden state h_{t-1} (your “notebook” so far), and computes a new hidden state

h_t . You can then use the hidden state h_t to make a prediction (e.g. the next track), or pass it onto the next step.

Plain RNNs struggle as over a long period of time (after many epochs), when you train by back-propagating errors through many time steps, the gradients can shrink to almost zero. This makes it hard for the model to learn long-range dependencies (e.g. remembering a motif from 50 tracks ago). They also face issues of exploding gradients, where gradients blow up and make training unstable.

GRU's essentially have little mechanisms that learn to control how much information to keep or throw away at each step. A GRU has two gates, which can be seen as two valves that regulate the flow of information into memory:

- Update gate (z_t) $\implies$ "How much of my new candidate info should I write into memory versus keep from the past?"
- Reset gate (r_t) $\implies$ "When reading the past, should I 'forget' some of it before mixing in new info?"

Here is what the GRU "sees" at each step

- Input: At time step t , the GRU receives a 9-D audio-feature vector x_t for one track in the playlist context (danceability, energy, loudness, tempo, etc.)
- Masking: If a playlist has fewer than 100 tracks, those "missing" positions are padded and ignored—so every sequence fed to the GRU is exactly length 100, but only the real tracks actually update the hidden state

The GRU processes the sequence like this:

- Hidden state h_{t-1} carries all information from the previous $t - 1$ tracks
- Reset gate decides whether to "erase" parts of that history before mixing in the new x_t
- Update gate decides how much of the newly computed information to write into the new state h_t vs how much of the old state to keep

By the time you feed in the 100th track, the GRU's final hidden state h_{100} is a learned summary of the entire playlists recent listening "story" - which styles have dominated, how energy has fluctuated, and so on. h_{100} is a fixed-length (64-dimensional) vector capturing the "theme" of the preceding tracks. In effect, it has learned to encode things like

- "This listener has been on a high-energy dance kick," or
- "They just played an acoustic ballad, then a podcast snippet," or
- "They mix fast and slow tracks fairly evenly"

Because GRUs learn these patterns end-to-end, similar playlists will map to nearby points in this 64-dimensional hidden space.

Once you have h_{100} , you bring in your candidate track's own 9-dimensional feature vector c . You then

- Concatenate these two into a 73-dim vector
- Pass it through a small feed-forward network that learns to score how well this candidate “fits” the preceding context

If the candidate truly was the next track in the playlist, the model should output a probability close to 1; if it's a random track from the same cluster, closer to 0.

3.4 GRU Neuralnet Results

After training, we evaluate the GRU sequence model on held-out playlists by posing a series of “next-track” questions: for each playlist in the test split, we present the true next track (a positive example) alongside an equal number (or a fixed negative ratio) of “distractor” tracks sampled from the same cluster. The model outputs a probability score for each candidate indicating how likely it is to be the actual next track. A high score means the model thinks this track to be a potential recommender track; a low score means the model thinks it is a distractor track. We then compare those scores to the ground-truth labels (1 for the real next-track, 0 for negatives) to compute summary metrics. We split our dataset of $\sim N$ samples into the standard 80% train and 20% test.

On the test set, we call

```
1 proba_te = model.predict({"seq_in": X_seq_te, "cand_in":  
    X_cand_te})
```

obtaining a probability $p_i \in [0, 1]$ for each candidate sample. We then feed the vector of true labels (y_i) and predicted scores (p_i) into metric functions.

ROC - AUC (Receiver Operating Characteristic – Area Under Curve)

- Plots the true positive rate vs. the false positive rate as we sweep the decision threshold from 0 to 1
- AUC is the probability that a randomly chosen positive example is ranked higher than a randomly chosen negative one
- Range: 0.5 = random guessing; 1.0 = perfect ranking
- For context, a score above 0.85 is considered high accuracy, while a score of 0.75 to 0.85 suggest moderate accuracy. Below 0.75 usually indicates low accuracy
- We use it because it is threshold independent, meaning that it summarizes the model's ability to separate positives from negatives regardless of where the cut-off (probability threshold) is set

MAP (Mean Average Precision)

- Also known as Average Precision in binary settings. It is the area under the precision-recall curve, which plots precision vs. recall as we vary the threshold
- Precision = fraction of predicted “next” tracks that are actually correct; Recall = fraction of true “next” tracks that we successfully retrieve
- We use it because in imbalanced tasks (far more negatives than positives), precision-recall metrics often give a more informative picture than ROC. MAP rewards models that rank true positives high, even if there are many easy negatives

Below we have included a table that shows our results across different sample sizes. Note that we could only go up to 50,000 playlists due to time constraints and RAM limitations - the system was extremely unstable beginning at 100,000 playlists.

# Playlists	ROC-AUC	MAP
1 000	0.515	0.346
10 000	0.540	0.351
30 000	0.672	0.470
50 000	0.708	0.507

- With 1,000 playlists, the model barely outperforms chance (ROC-AUC ~ 0.5), indicating it doesn’t see enough varied sequences to learn reliable patterns
- Scaling to 10,000 brings a small gain, but still modest, as the model is data-hungry
- At 30,000, we cross a meaningful threshold: ROC-AUC jumps to 0.672, MAP to 0.470—showing the GRU is now capturing genuine sequential structure
- By 50,000, we see further improvement (ROC-AUC = 0.708, MAP = 0.507), suggesting diminishing but continuing returns to more data

The key takeaway is that the GRU’s ability to predict the next track improves substantially with more training examples. Early on (small N) overfits to noise and sees little signal; past 30k playlists, it has enough samples to determine playlist “vibes” and transitions to generalizing well.

4 Optimizations

To optimize our models, we chose to implement a GPU-based MiniBatchK-Means. GPU-based simply means that we are offloading the two heaviest

steps - distance computations and centroid updates - onto the GPU. Libraries like RAPIDS cuML leverage CUDA to launch thousands of small distance-to-centroid kernels in parallel, and use high-bandwidth GPU memory to stream minibatches. On paper, this is better because

- A single GPU can compute distances for millions of points to dozens of centroids simultaneously, whereas a CPU cores typically work on far fewer vectors at once
- GPU memory bandwidth (500 GB/s+) dwarfs typical DDR4 (20–30 GB/s), so loading feature arrays and writing centroid updates happens much faster

Together, this should speed up processing times magnitudes faster than the vanilla CPU-based MiniBatchKMeans, especially when running larger subsets of the dataset. However, due to RAM constraints, we were not able to see the full extent of the improvement.

In addition, we also used Optuna to optimize hyperparameter tuning for MiniBatchKMeans and the GRU neuralnet. Optuna is an open sourced Python framework that utilizes various algorithms to identify the optimal hyperparameter combination. One such algorithm is the bayesian hyperparameter optimization. Optuna can stop unpromising trails early based on intermediate metrics, functioning as a sort of pruning technique (similar to alpha beta pruning for graph based problems). Optuna is designed for large-scale optimization problems, making it especially useful for this dataset. Once again, due to RAM constraints, we were unable to full measure the improvements versus the baseline model.

4.1 Optimization Results MiniBatchKMeans

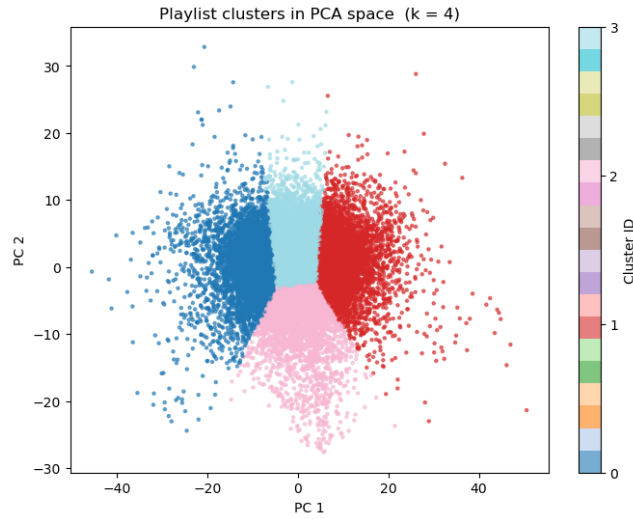


Figure 6: Playlist clusters in PCA space for Optimized (Optuna + GPU) version

Loadings on PC1 and PC2:		
	PC1	PC2
danceability_mean	0.000	-0.005
danceability_std	-0.000	0.001
energy_mean	0.008	-0.007
energy_std	-0.001	0.002
loudness_mean	0.151	-0.114
loudness_std	-0.048	0.047
speechiness_mean	0.000	0.000
speechiness_std	-0.000	0.000
acousticness_mean	-0.010	0.010
acousticness_std	-0.003	0.004
instrumentalness_mean	-0.003	0.000
instrumentalness_std	-0.002	-0.002
liveness_mean	0.001	-0.000
liveness_std	0.001	-0.001
valence_mean	0.002	-0.002
valence_std	0.000	0.000
tempo_mean	0.986	-0.036
tempo_std	0.055	0.992

Figure 7: Optimized PCA loadings for PC 1 and PC 2

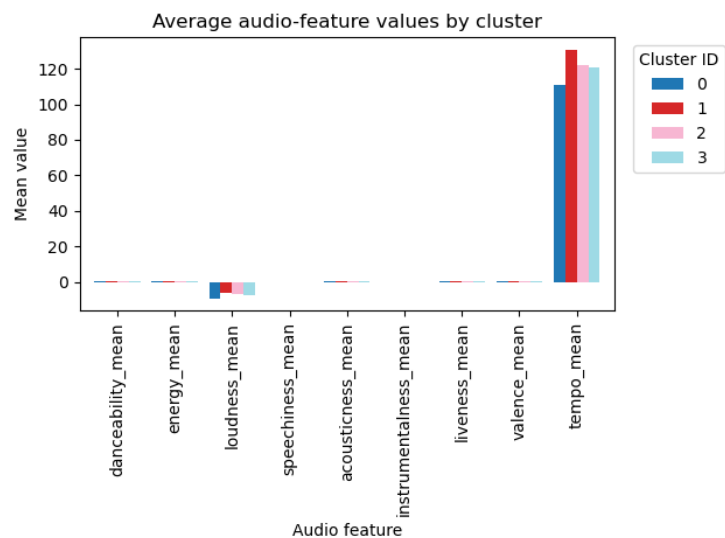


Figure 8: Optimized Cluster Centroids in Audio Feature Space

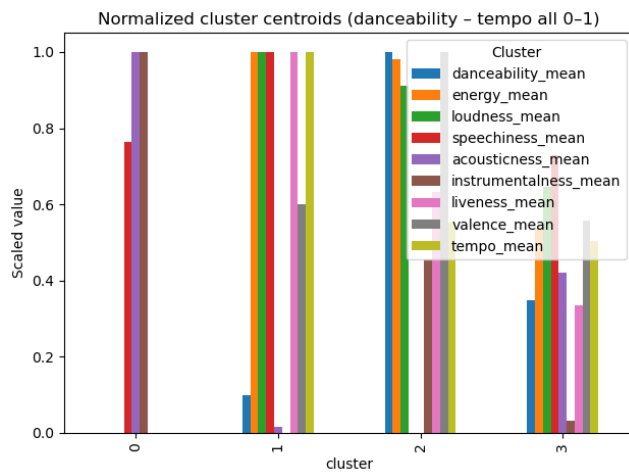


Figure 9: Optimized Normalized Cluster Centroids

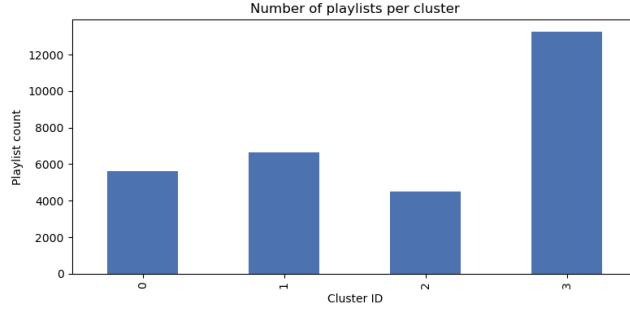


Figure 10: Playlist Count per Cluster with Optimization

Visual Comparison:

- The vanilla wedges (Figure 1) show four broad slices, but you can see a bit of “bleed” at the boundaries, particularly between the central (Cluster 0) and high-energy (Cluster 1) sectors
- GPU + Optuna (new Figure 6) tightens these wedges: the boundaries between clusters are crisper, with fewer stray points crossing from one slice into the next

Comparing the normalized centroids side by side:

- Cluster 1 (upbeat/dance) sees its normalized energy and tempo tick slightly higher, sharpening its gap from Cluster 0
- Cluster 2 (acoustic/folk) becomes a bit more extreme on acousticness and instrumentality, pulling it farther from the mid-tempo groups
- Clusters 0 and 3 pull slightly inward, reflecting how Optuna pushed the decision boundaries to maximize separation

Comparing Cluster-Quality Metrics:

Metric	Vanilla MBKM	GPU + Optuna MBKM	Δ (relative)
Silhouette	0.2821	0.2939	+4.2%
Davies-Bouldin	1.0424	1.0220	-1.9%
Best-k	4	4	—

Table 1: Comparison between Vanilla MiniBatchKMeans and GPU + Optuna MiniBatchKMeans

- Silhouette score went up from 0.282 to 0.294, indicating playlists sit closer to their own centroid and farther from their nearest neighbor cluster
- Davies-Bouldin index fell from 1.042 to 1.022, confirming clusters are more compact and better separated on average

- the δ (relative) shows the relative change of each metric when you move from the vanilla CPU to the GPU MiniBatchKMeans + Optuna tuned version. It is calculated as $\Delta = \frac{(\text{GPU+Optuna value}) - (\text{Vanilla value})}{\text{Vanilla value}} \times 100\%$. Thus, we can see that both metrics are showing positive growth, indicating that the optimizations were successful.

4.2 GRU Optimization Results

Playlists	Baseline ROC-AUC	Baseline MAP	Optuna ROC-AUC	Optuna MAP
1,000	0.515	0.346	0.460	0.306
10,000	0.540	0.351	0.614	0.407
30,000	0.672	0.470	0.693	0.495
50,000	0.708	0.507	0.708	0.507

Table 2: GRU sequence model performance before and after Optuna tuning, evaluated on different dataset sizes.

Metrics were not taken for playlist sizes above 50,000 on both the Optuna and Baseline versions as there was not enough time; the most we could manage was 100,000 (higher than this the RAM would overload), but the program was extremely unstable (especially for the GRU portion), taking over 10 hours each and crashing more than half the time. Eventually, we ran out of time to have the model finish before the presentation date. For 10,000 playlists,

- ROC-AUC jumps from 0.540 $\rightarrow$ 0.614 (+0.074), a 7.4 pp gain in ranking quality
- MAP rises from 0.351 $\rightarrow$ 0.407 (+0.056), a 5.6 pp boost in precision-recall area
- This large uplift shows that hyperparameter tuning is decently fruitful even when training data is modest

For 30,000 playlists,

- ROC-AUC improves from 0.672 $\rightarrow$ 0.693 (+0.021, +3.1 pp)
- MAP improves from 0.470 $\rightarrow$ 0.495 (+0.025, +2.5 pp)
- Gains persist but are smaller, reflecting diminishing returns as the model already has ample data (unknown whether this trend will continue into higher amounts of playlists)

Overall, it is clear that Optuna induces improvement in at least two metric stats, ROC-AUC and MAP.

5 Phase 1 (unsupervised) results for 50k playlists

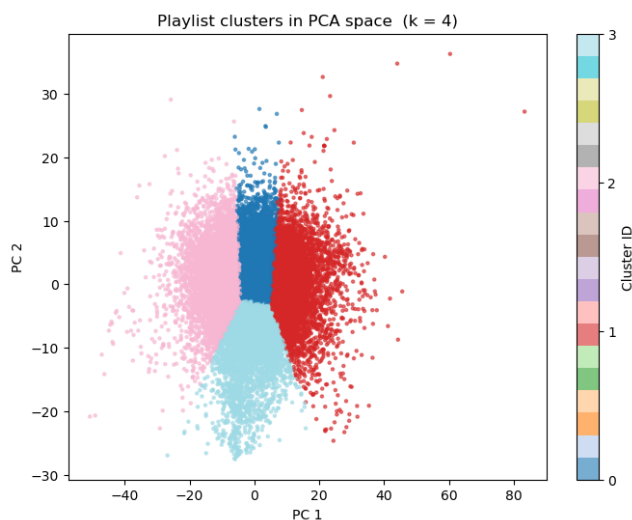


Figure 11: Playlist clusters in PCA space for Optimized (Optuna + GPU) version for 50k playlists

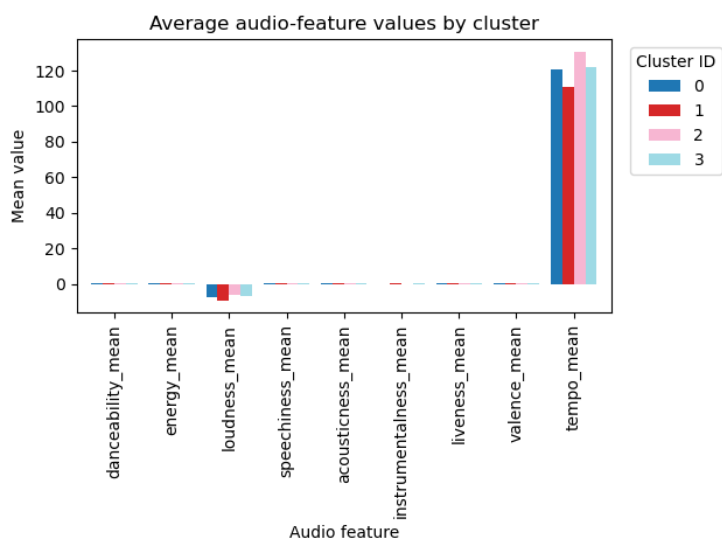


Figure 12: Optimized Cluster Centroids in Audio Feature Space at 50k playlists

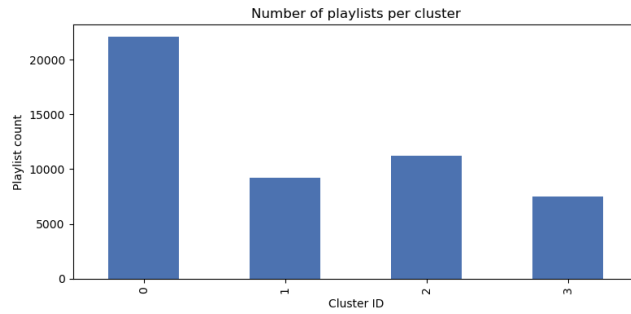


Figure 13: Playlist Count per Cluster with Optimization at 50k playlists

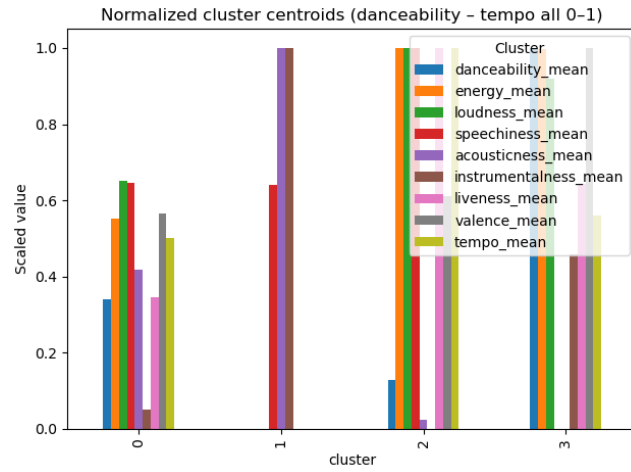


Figure 14: Optimized Normalized Cluster Centroids at 50k playlists

6 Results on larger dataset (100k playlists +)

To be filled in if model trains successfully - model crashed : (